

“DBMS LAB FILES”

SUBMITTED BY

NAME=SUDARSHAN DHARAM

PRN=2124UCSM1079

{Data Base Management System }

UNDER THE GUIDANCE OF

MR. Utkarsh Varshney Sir



Department of

“Computer Science and Engineering”

SANJIVANI RURAL EDUCATION SOCIETY

SANJIVANI UNIVERSITY

KOPARGAON – 423603,

DIST : AHMEDNAGAR 2024-25

INDEX

Sr. No.	Practical Title	Date	Signature	Page No.
1	Install and Set Up MySQL. Create Database and Table for Employee Details. Perform INSERT & DELETE.			
2	Create Student Table. Perform INSERT, UPDATE, DELETE, and SELECT Operations.			
3	Create Employee Table with Data Types. Insert Data and Perform Queries.			
4	Create Employee Table with Constraints (Primary Key, Foreign Key, Unique). Test with Valid/Invalid Data.			
5	Create Customer Table with NOT NULL, CHECK, DEFAULT Constraints. Test with Valid/Invalid Data.			
6	Use DDL and DML Commands. Perform Insert, Update, Delete, and Data Retrieval Using SELECT Queries.			
7	Create Sales Table. Use Aggregate Functions: COUNT, SUM, AVG, MIN, MAX.			
8	Use INNER JOIN, LEFT JOIN, and RIGHT JOIN on Customers and Orders Tables.			

Lab File – Practical 1

AIM :- Install and set up MySQL. Create a database and a table to store employee details.

- **CODE FOR INSERTION :-**

```
CREATE DATABASE PRACTICAL1;

USE PRACTICAL1;

CREATE TABLE EMPLOYEE(
id int auto_increment primary key,
name varchar(50) not null,
departement varchar(50) not null,
position varchar(50) not null,
salary decimal(10,2),
joindate date not null);

select * from EMPLOYEE;

insert into EMPLOYEE (name, departement, position, salary, joindate)
VALUES

('sudarshan', 'COMPUTER', 'MANAGER', '10000.00', '2020-03-04'),
('ALEX', 'CYBER', 'MANAGER', '10000.00', '2020-03-04'),
('ROBIN', 'COMPUTER', 'EMPLOYEE', '1000.00', '2024-01-12'),
('ROCKEY', 'COMPUTER', 'EMPLOYEE', '1000.00', '2023-06-14'),
('KAILY', 'CYBER', 'EMPLOYEE', '1000.00', '2020-03-04');

select * from EMPLOYEE;
```

- **OUTPUT :-**

The screenshot shows the OneCompiler MySQL interface. The left pane contains the following SQL code:

```

-- CREATE TABLE EMPLOYEE(
-- id int auto_increment primary key,
-- name varchar(50) not null,
-- departement varchar(50) not null,
-- position varchar(50) not null,
-- salary decimal(10,2),
-- joindate date not null);
-- insert into EMPLOYEE (name, departement, position, salary, joindate)
-- values
-- ('sudarshan', 'COMPUTER', 'MANAGER', '10000.00', '2020-03-04'),
-- ('ALEX', 'CYBER', 'MANAGER', '10000.00', '2020-03-04'),
-- ('ROCKY', 'COMPUTER', 'EMPLOYEE', '10000.00', '2020-03-11'),
-- ('ROCKY', 'COMPUTER', 'EMPLOYEE', '10000.00', '2020-03-11'),
-- ('KALLY', 'CYBER', 'EMPLOYEE', '10000.00', '2020-03-04');
-- select * from EMPLOYEE;
  
```

The right pane shows the output of the query, which is a table with 5 rows and 5 columns: id, name, departement, position, and salary. The output is as follows:

id	name	departement	position	salary	joindate
1	sudarshan	COMPUTER	MANAGER	10000.00	2020-03-04
2	ALEX	CYBER	MANAGER	10000.00	2020-03-04
3	ROCKY	COMPUTER	EMPLOYEE	10000.00	2020-03-11
4	ROCKY	COMPUTER	EMPLOYEE	10000.00	2020-03-11
5	KALLY	CYBER	EMPLOYEE	10000.00	2020-03-04

- **CODE FOR DELETION :-**

```

CREATE TABLE EMPLOYEE(
id int auto_increment primary key,
name varchar(50) not null,
departement varchar(50) not null,
position varchar(50) not null,
salary decimal(10,2),
joindate date not null);
select * from EMPLOYEE;
insert into EMPLOYEE (name, departement, position, salary, joindate)
VALUES
('sudarshan', 'COMPUTER', 'MANAGER', '10000.00', '2020-03-04'),
('ALEX', 'CYBER', 'MANAGER', '10000.00', '2020-03-04'),
  
```

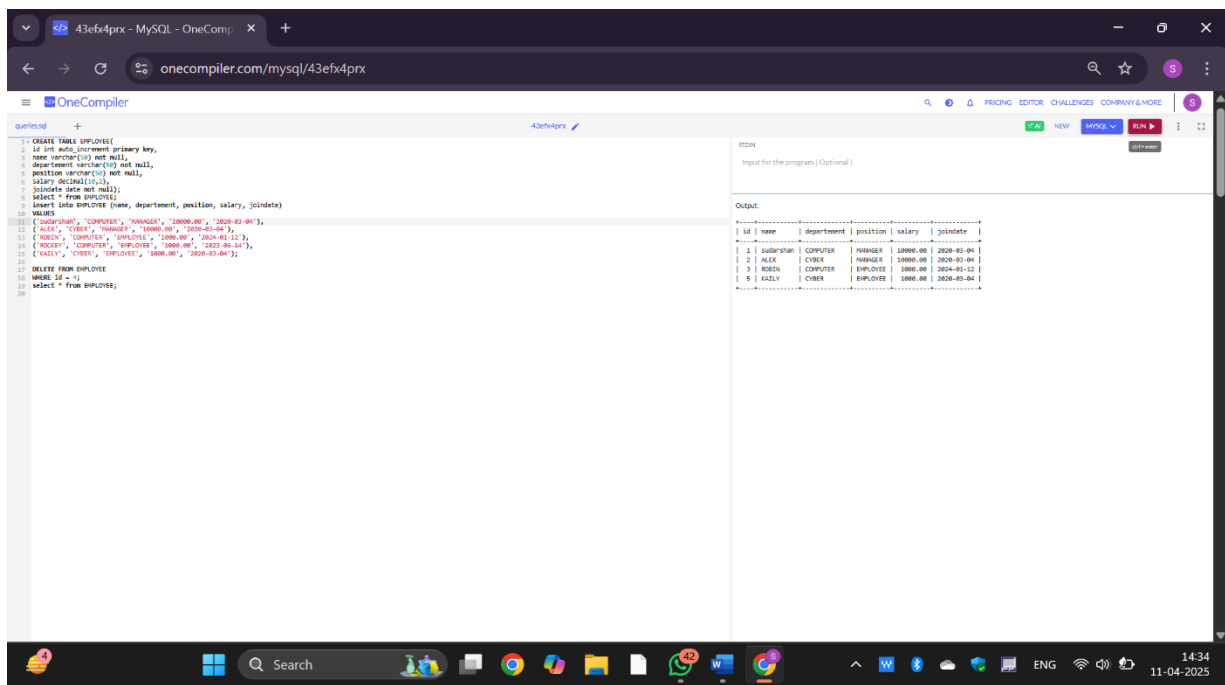
```
('ROBIN', 'COMPUTER', 'EMPLOYEE', '1000.00', '2024-01-12'),  
('ROCKEY', 'COMPUTER', 'EMPLOYEE', '1000.00', '2023-06-14'),  
('KAILY', 'CYBER', 'EMPLOYEE', '1000.00', '2020-03-04');
```

DELETE FROM EMPLOYEE

WHERE id = 4;

select * from EMPLOYEE;

OUTPUT :-



The screenshot shows the OneCompiler MySQL IDE interface. The left pane contains the following SQL code:

```
1 CREATE TABLE EMPLOYEE(  
2   id int auto_increment primary key,  
3   name varchar(100) not null,  
4   department varchar(50) not null,  
5   position varchar(50) not null,  
6   salary decimal(10,2),  
7   joindate date not null);  
8  
9 select * from EMPLOYEE;  
10  
11 insert into EMPLOYEE (name, department, position, salary, joindate)  
12  
13 VALUES  
14 ('SUDARSHAN', 'COMPUTER', 'MANAGER', '10000.00', '2020-03-04'),  
15 ('ALEX', 'CYBER', 'MANAGER', '10000.00', '2020-03-04'),  
16 ('ROCKY', 'COMPUTER', 'EMPLOYEE', '1000.00', '2024-01-12'),  
17 ('ROCKEY', 'COMPUTER', 'EMPLOYEE', '1000.00', '2023-06-14'),  
18 ('KAILY', 'CYBER', 'EMPLOYEE', '1000.00', '2020-03-04');  
19  
20 DELETE FROM EMPLOYEE  
21 WHERE id = 4;  
22  
23 select * from EMPLOYEE;  
24
```

The right pane shows the output of the queries:

Input for the program (Optional):

Output:

id	name	department	position	salary	joindate
1	SUDARSHAN	COMPUTER	MANAGER	10000.00	2020-03-04
2	ALEX	CYBER	MANAGER	10000.00	2020-03-04
3	ROCKY	COMPUTER	EMPLOYEE	1000.00	2024-01-12
5	KAILY	CYBER	EMPLOYEE	1000.00	2020-03-04

Lab File – Practical 2

Aim: Create a table for storing student information. Insert sample data and perform basic Operations: INSERT, UPDATE, DELETE, and SELECT.

- **CODE FOR TABLE CREATION :-**

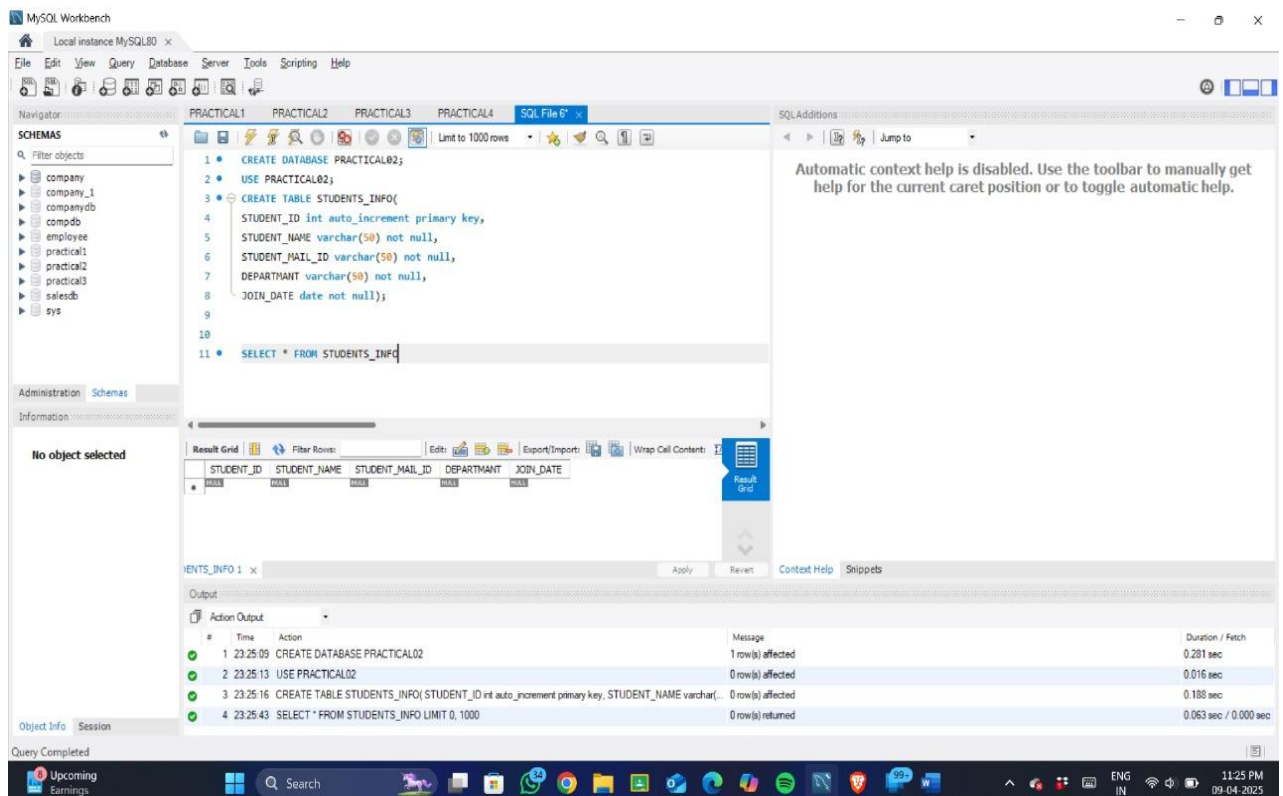
```
CREATE DATABASE PRACTICAL2;
```

```
USE PRACTICAL2;
```

```
CREATE TABLE STUDENTS_INFO(  
STUDENT_ID int auto_increment primary key,  
STUDENT_NAME varchar(50) not null,  
STUDENT_MAIL_ID varchar(50) not null,  
DEPARTMANT varchar(50) not null,  
JOIN_DATE date not null);
```

```
SELECT * FROM STUDENTS_INFO
```

- **OUTPUT :-**



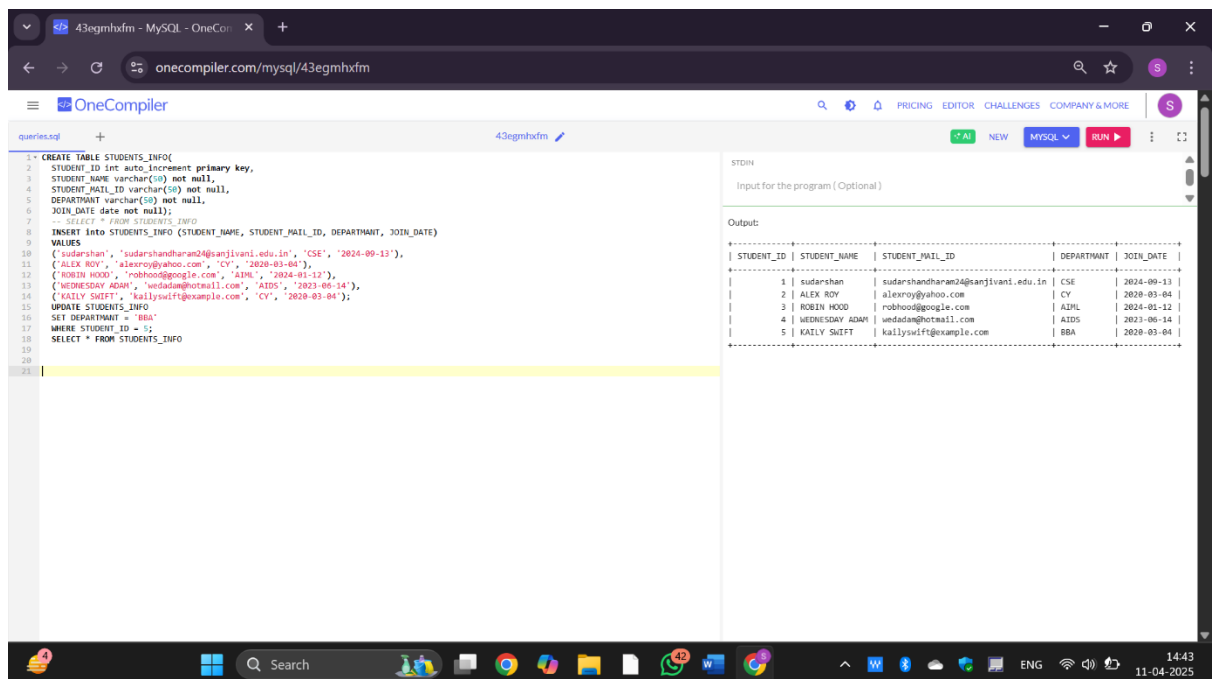
- **CODE FOR SOME SAMPLE DATA :-**

- **CREATE TABLE STUDENTS_INFO(**

- STUDENT_ID int auto_increment primary key,
- STUDENT_NAME varchar(50) not null,
- STUDENT_MAIL_ID varchar(50) not null,
- DEPARTMANT varchar(50) not null,
- JOIN_DATE date not null);
- -- SELECT * FROM STUDENTS_INFO
- INSERT into STUDENTS_INFO (STUDENT_NAME, STUDENT_MAIL_ID, DEPARTMANT, JOIN_DATE)
- VALUES
- ('sudarshan', 'sudarshandharam24@sanjivani.edu.in', 'CSE', '2024-09-13'),
- ('ALEX ROY', 'alexroy@yahoo.com', 'CY', '2020-03-04'),
- ('ROBIN HOOD', 'robhood@google.com', 'AIML', '2024-01-12'),
- ('WEDNESDAY ADAM', 'wedadam@hotmail.com', 'AIDS', '2023-06-14'),
- ('KAILY SWIFT', 'kailyswift@example.com', 'CY', '2020-03-04');

- **OUTPUT:-**

- **CODE FOR UPDATE THE TABLE :-**



The screenshot shows the OneCompiler MySQL interface. The left pane contains the following SQL code:

```
1 CREATE TABLE STUDENTS_INFO(
2   STUDENT_ID int auto_increment primary key,
3   STUDENT_NAME varchar(50) not null,
4   STUDENT_MAIL_ID varchar(50) not null,
5   DEPARTMANT varchar(50) not null,
6   JOIN_DATE date not null);
7 -- SELECT * FROM STUDENTS_INFO
8 INSERT into STUDENTS_INFO (STUDENT_NAME, STUDENT_MAIL_ID, DEPARTMANT, JOIN_DATE)
9 VALUES
10 ('sudarshan', 'sudarshandharam24@sanjivani.edu.in', 'CSE', '2024-09-13'),
11 ('ALEX ROY', 'alexroy@yahoo.com', 'CY', '2020-03-04'),
12 ('ROBIN HOOD', 'robhood@google.com', 'AIML', '2024-01-12'),
13 ('WEDNESDAY ADAM', 'wedadam@hotmail.com', 'AIDS', '2023-06-14'),
14 ('KAILY SWIFT', 'kailyswift@example.com', 'CY', '2020-03-04');
15 UPDATE STUDENTS_INFO
16 SET DEPARTMANT = 'BBA'
17 WHERE STUDENT_ID = 5;
18 SELECT * FROM STUDENTS_INFO
```

The right pane shows the output of the queries:

Output:

STUDENT_ID	STUDENT_NAME	STUDENT_MAIL_ID	DEPARTMANT	JOIN_DATE
1	sudarshan	sudarshandharam24@sanjivani.edu.in	CSE	2024-09-13
2	ALEX ROY	alexroy@yahoo.com	CY	2020-03-04
3	ROBIN HOOD	robhood@google.com	AIML	2024-01-12
4	WEDNESDAY ADAM	wedadam@hotmail.com	AIDS	2023-06-14
5	KAILY SWIFT	kailyswift@example.com	BBA	2020-03-04

- UPDATE STUDENTS_INFO
- SET DEPARTMANT = 'BBA'
- WHERE STUDENT_ID = 5;
- SELECT * FROM STUDENTS_INFO

OUTPUT :-

The screenshot shows the OneCompiler MySQL interface. The left pane contains the following SQL code:

```
1 CREATE TABLE STUDENTS_INFO(  
2   STUDENT_ID int auto-increment primary key,  
3   STUDENT_NAME varchar(50) not null,  
4   STUDENT_MAIL_ID varchar(50) not null,  
5   DEPARTMENT varchar(50) not null,  
6   JOIN_DATE date not null);  
7  
8 -- SELECT * FROM STUDENTS_INFO  
9 INSERT INTO STUDENTS_INFO (STUDENT_NAME, STUDENT_MAIL_ID, DEPARTMENT, JOIN_DATE)  
10 VALUES  
11 ('Sudarshan', 'sudarshandharan24@anjiivani.edu.in', 'CSE', '2024-09-13'),  
12 ('ALEX ROY', 'alexroy@yahoo.com', 'CY', '2020-03-04'),  
13 ('ROBIN HOOD', 'robhoo@google.com', 'AIZH', '2024-01-12'),  
14 ('WEDNESDAY ADAM', 'wedadag@hotmail.com', 'AIDS', '2023-06-14'),  
15 ('KALLY SWIFT', 'kallyswift@example.com', 'CV', '2020-03-04');  
16  
17 UPDATE STUDENTS_INFO  
18 SET DEPARTMENT = 'BBA'  
19 WHERE STUDENT_ID = 5;  
20  
21 SELECT * FROM STUDENTS_INFO
```

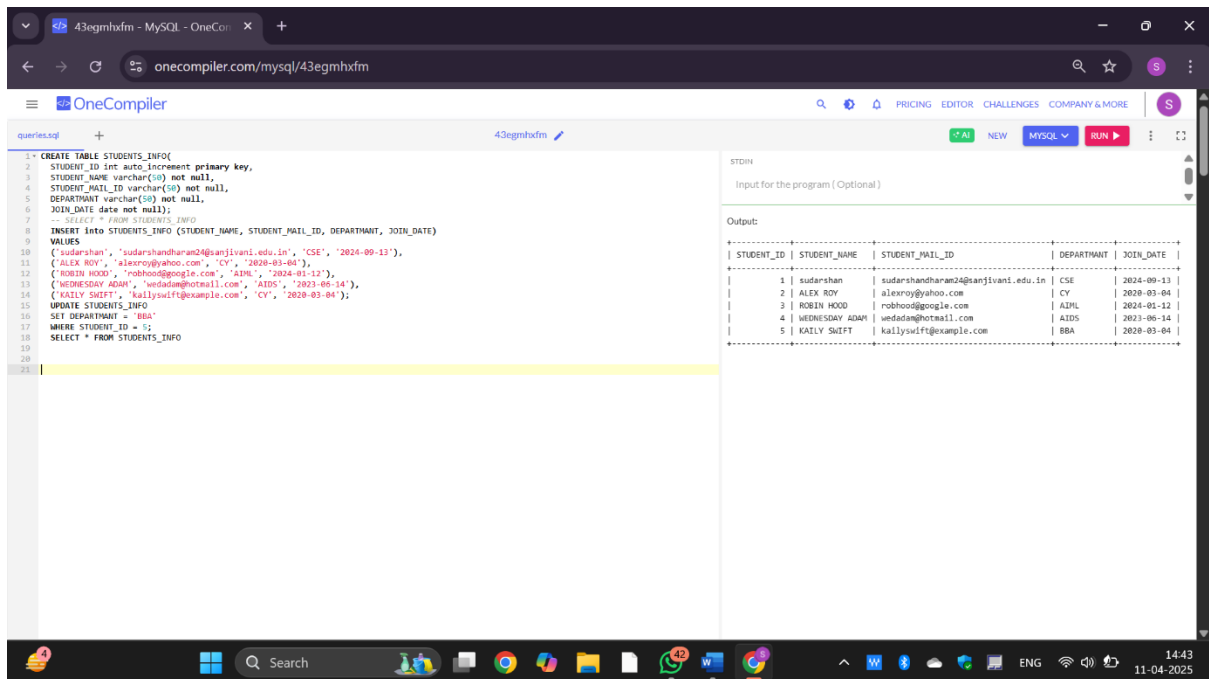
The right pane shows the output of the SQL code, which is a table with 5 rows and 5 columns: STUDENT_ID, STUDENT_NAME, STUDENT_MAIL_ID, DEPARTMENT, and JOIN_DATE.

STUDENT_ID	STUDENT_NAME	STUDENT_MAIL_ID	DEPARTMENT	JOIN_DATE
1	Sudarshan	sudarshandharan24@anjiivani.edu.in	CSE	2024-09-13
2	ALEX ROY	alexroy@yahoo.com	CY	2020-03-04
3	ROBIN HOOD	robhoo@google.com	AIZH	2024-01-12
4	WEDNESDAY ADAM	wedadag@hotmail.com	AIDS	2023-06-14
5	KALLY SWIFT	kallyswift@example.com	BBA	2020-03-04

- **CODE FOR DELETE THE RECORD FROM THE TABLE :-**

```
DELETE FROM STUDENTS_INFO  
WHERE STUDENT_ID = 3;  
SELECT * FROM STUDENTS_INFO
```


OUTPUT :-



The screenshot shows the OneCompiler web interface for MySQL. The left pane contains a SQL query to create a table, insert data, and update a department. The right pane shows the output of the query, which is a table with 5 columns: STUDENT_ID, STUDENT_NAME, STUDENT_MAIL_ID, DEPARTMENT, and JOIN_DATE. The output table contains 5 rows of data.

```
1. CREATE TABLE STUDENTS_INFO(  
2.   STUDENT_ID int auto-increment primary key,  
3.   STUDENT_NAME varchar(50) not null,  
4.   STUDENT_MAIL_ID varchar(50) not null,  
5.   DEPARTMENT varchar(50) not null,  
6.   JOIN_DATE date not null);  
7. -- SELECT * FROM STUDENTS_INFO  
8. INSERT INTO STUDENTS_INFO (STUDENT_NAME, STUDENT_MAIL_ID, DEPARTMENT, JOIN_DATE)  
9. VALUES  
10. ('sudarshan', 'sudarshandharam24@sanjivani.edu.in', 'CSE', '2024-09-13'),  
11. ('ALEX ROY', 'alexroy@yahoo.com', 'CY', '2020-03-04'),  
12. ('ROBIN HOOD', 'robhoo@google.com', 'ADPL', '2024-01-12'),  
13. ('WEDNESDAY ADAM', 'wedadam@hotmail.com', 'AIDS', '2023-06-14'),  
14. ('KALY SWIFT', 'kallyswift@example.com', 'CY', '2020-03-04');  
15. UPDATE STUDENTS_INFO  
16. SET DEPARTMENT = 'BBA'  
17. WHERE STUDENT_ID = 5;  
18. SELECT * FROM STUDENTS_INFO
```

STUDENT_ID	STUDENT_NAME	STUDENT_MAIL_ID	DEPARTMENT	JOIN_DATE
1	sudarshan	sudarshandharam24@sanjivani.edu.in	CSE	2024-09-13
2	ALEX ROY	alexroy@yahoo.com	CY	2020-03-04
3	ROBIN HOOD	robhoo@google.com	ADPL	2024-01-12
4	WEDNESDAY ADAM	wedadam@hotmail.com	AIDS	2023-06-14
5	KALY SWIFT	kallyswift@example.com	BBA	2020-03-04

Lab File – Practical 3

AIM :- Create a table with columns for EmployeeID, Name, Salary, JoiningDate, and ActiveStatus using different data types. Insert sample data and perform queries to manipulate and retrieve data.

- **CODE FOR CREATION OF TABLE :-**

```
CREATE DATABASE PRACTICAL3;
```

```
USE PRACTICAL3;
```

```
CREATE TABLE Employees (
```

```
EmployeeID INT PRIMARY KEY,
```

```
Name VARCHAR(100),
```

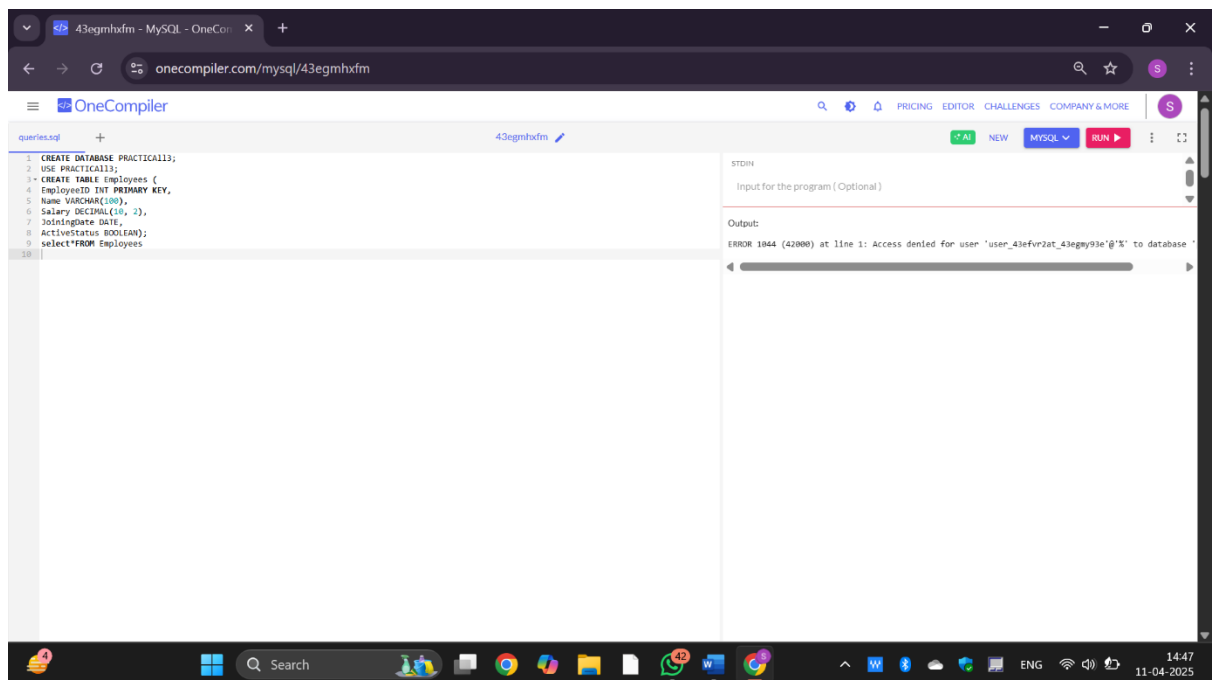
```
Salary DECIMAL(10, 2),
```

```
JoiningDate DATE,
```

```
ActiveStatus BOOLEAN);
```

```
select*FROM Employees
```

- **OUTPUT :-**



- **CODE FOR INSERTION OF SAMPLE DATA :-**

INSERT INTO Employees (EmployeeID, Name, Salary, JoiningDate, ActiveStatus)

VALUES

(101, 'sudrashna dharam', '85000.00', '2023-02-18', TRUE),

(102, 'Avery Knight', 55000.00, '2020-06-15', FALSE),

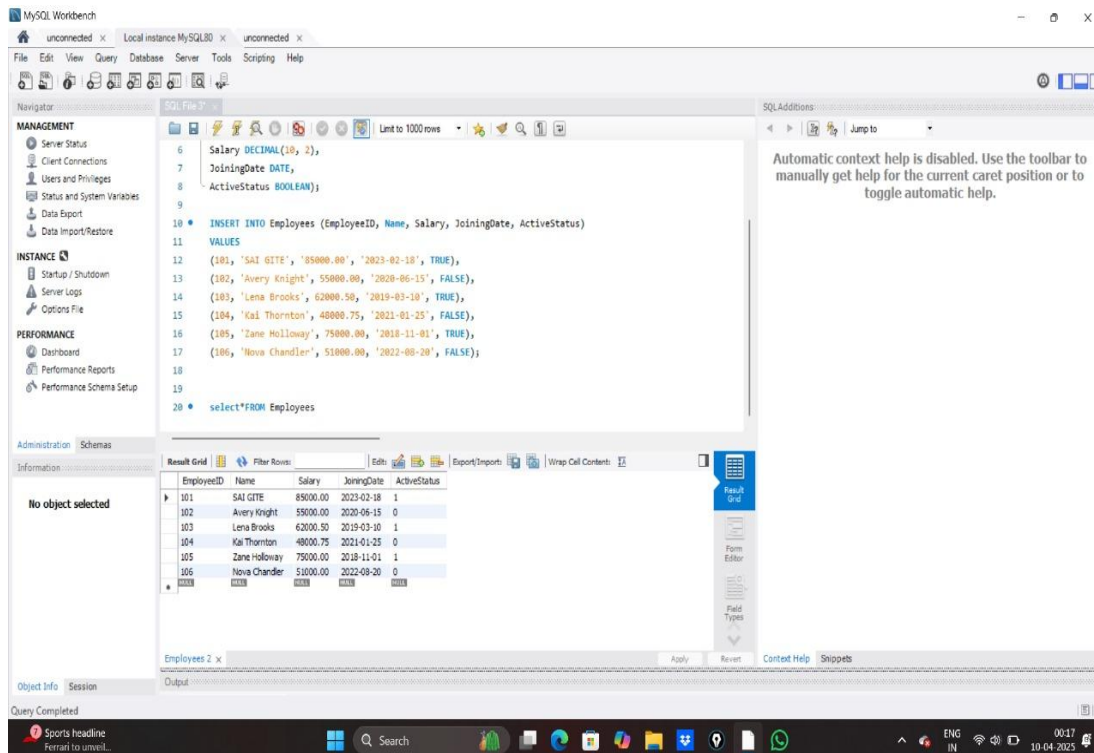
(103, 'Lena Brooks', 62000.50, '2019-03-10', TRUE),

(104, 'Kai Thornton', 48000.75, '2021-01-25', FALSE),

(105, 'Zane Holloway', 75000.00, '2018-11-01', TRUE),

(106, 'Nova Chandler', 51000.00, '2022-08-20', FALSE);

SELECT * FROM Employees



- **OUTPUT :-**

- **CODE FOR PERFORMING QUERIES TO MANIPULATE AND RETRIEVE DATA :-**

1. SELECT * FROM Employees

● OUTPUT :-

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following code:

```
6 Salary DECIMAL(10, 2),
7 JoiningDate DATE,
8 ActiveStatus BOOLEAN);
9
10 INSERT INTO Employees (EmployeeID, Name, Salary, JoiningDate, ActiveStatus)
11 VALUES
12 (101, 'SAI GITE', '85000.00', '2023-02-18', TRUE),
13 (102, 'Avery Knight', '55000.00', '2020-06-15', FALSE),
14 (103, 'Lena Brooks', '62000.50', '2019-03-10', TRUE),
15 (104, 'Kai Thornton', '48000.75', '2021-01-25', FALSE),
16 (105, 'Zane Holloway', '75000.00', '2018-11-01', TRUE),
17 (106, 'Nova Chandler', '51000.00', '2022-08-20', FALSE);
18
19
20 select * FROM Employees
```

The Result Grid shows the output of the query:

EmployeeID	Name	Salary	JoiningDate	ActiveStatus
101	SAI GITE	85000.00	2023-02-18	1
102	Avery Knight	55000.00	2020-06-15	0
103	Lena Brooks	62000.50	2019-03-10	1
104	Kai Thornton	48000.75	2021-01-25	0
105	Zane Holloway	75000.00	2018-11-01	1
106	Nova Chandler	51000.00	2022-08-20	0

2. SELECT * FROM Employees WHERE Salary > 60000 ;

OUTPUT :-

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following code:

```
5 EmployeeID INT PRIMARY KEY,
6 Name VARCHAR(100),
7 Salary DECIMAL(10, 2),
8 JoiningDate DATE,
9 ActiveStatus BOOLEAN
10 );
11
12 INSERT INTO Employees (EmployeeID, Name, Salary, JoiningDate, ActiveStatus)
13 VALUES
14 (101, 'SAI GITE', '85000.00', '2023-02-18', TRUE),
15 (102, 'Avery Knight', '55000.00', '2020-06-15', FALSE),
16 (103, 'Lena Brooks', '62000.50', '2019-03-10', TRUE),
17 (104, 'Kai Thornton', '48000.75', '2021-01-25', FALSE),
18 (105, 'Zane Holloway', '75000.00', '2018-11-01', TRUE),
19 (106, 'Nova Chandler', '51000.00', '2022-08-20', FALSE);
20
21 -- Correct query
22 SELECT * FROM Employees WHERE Salary > 60000;
```

The Result Grid shows the output of the query:

EmployeeID	Name	Salary	JoiningDate	ActiveStatus
101	SAI GITE	85000.00	2023-02-18	1
103	Lena Brooks	62000.50	2019-03-10	1
105	Zane Holloway	75000.00	2018-11-01	1

5.UPDATE Employees

SET ActiveStatus = TRUE

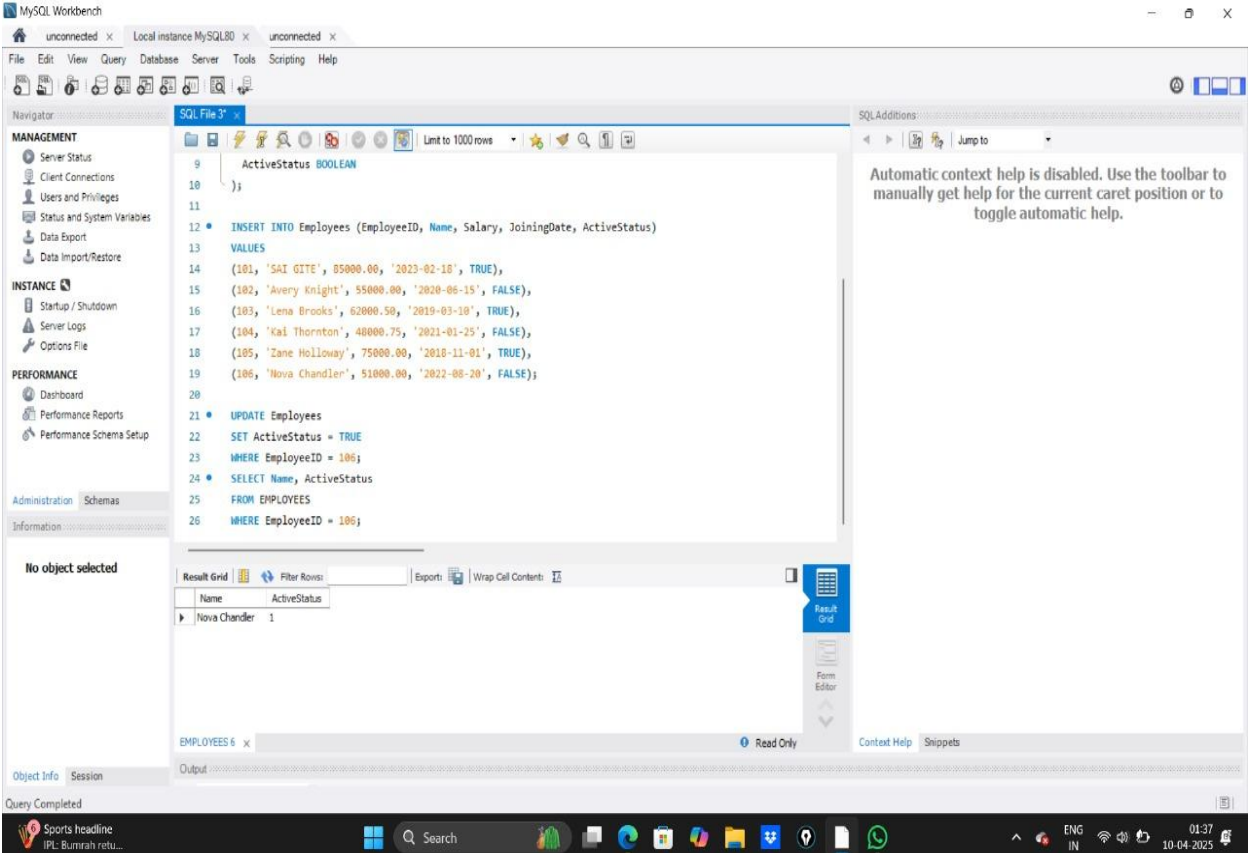
WHERE EmployeeID = 106;

SELECT Name, ActiveStatus

FROM EMPLOYEES

WHERE EmployeeID = 106;

• OUTPUT :-



The screenshot displays the MySQL Workbench interface. The SQL Editor contains the following queries:

```
9      ActiveStatus BOOLEAN
10    }
11
12 • INSERT INTO Employees (EmployeeID, Name, Salary, JoiningDate, ActiveStatus)
13   VALUES
14   (101, 'SAI GITE', 85000.00, '2023-02-10', TRUE),
15   (102, 'Avery Knight', 55000.00, '2020-06-15', FALSE),
16   (103, 'Lena Brooks', 62000.50, '2019-03-10', TRUE),
17   (104, 'Kai Thornton', 40000.75, '2021-01-25', FALSE),
18   (105, 'Zane Holloway', 75000.00, '2018-11-01', TRUE),
19   (106, 'Nova Chandler', 51000.00, '2022-08-20', FALSE);
20
21 • UPDATE Employees
22   SET ActiveStatus = TRUE
23   WHERE EmployeeID = 106;
24 • SELECT Name, ActiveStatus
25   FROM EMPLOYEES
26   WHERE EmployeeID = 106;
```

The Result Grid shows the output of the final query:

Name	ActiveStatus
Nova Chandler	1

The bottom status bar indicates "Query Completed".

Lab File – Practical 4

Aim: Create a table to store employee information with constraints like Primary Key, Foreign Key, and Unique.

Code :

```
CREATE TABLE Department (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50) UNIQUE  
);  
  
-- Create Employee Table  
  
CREATE TABLE Employee (  
    EmpID INT PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) UNIQUE,  
    Salary DECIMAL(10,2) CHECK (Salary > 0),  
    DeptID INT,  
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
);  
  
INSERT INTO Department (DeptID, DeptName) VALUES (1, 'HR');  
INSERT INTO Department (DeptID, DeptName) VALUES (2, 'IT');  
  
INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)
```

```
VALUES (101, 'sudarshan', 'sudarshan.d@example.com', 50000.00, 1);
```

```
INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)
```

```
VALUES (102, 'PRATIK', 'bob@example.com', 60000.00, 2);
```

```
INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)
```

```
VALUES (103, 'Charlie', 'charlie@example.com', 55000.00, 1);
```

```
INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)
```

```
VALUES (104, 'David', 'david@example.com', 45000.00, 2);
```

```
INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)
```

```
VALUES (105, 'Frank', 'frank@example.com', 40000.00, 1);
```

```
SELECT * FROM Employee;
```

```
SELECT * FROM Department;
```

Output :

The screenshot shows the OneCompiler MySQL interface. The left pane contains the following SQL code:

```
1 CREATE TABLE Department (  
2   DeptID INT PRIMARY KEY,  
3   DeptName VARCHAR(50) UNIQUE  
4 );  
5  
6 Create Employee Table  
7 CREATE TABLE Employee (  
8   EmpID INT PRIMARY KEY,  
9   Name VARCHAR(100) NOT NULL,  
10  Email VARCHAR(100) UNIQUE,  
11  Salary DECIMAL(10,2) CHECK (Salary > 0),  
12  DeptID INT,  
13  FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
14 );  
15 INSERT INTO Department (DeptID, DeptName) VALUES (1, 'HR');  
16 INSERT INTO Department (DeptID, DeptName) VALUES (2, 'IT');  
17 INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)  
18 VALUES (101, 'sudarshan', 'sudarshan.d@example.com', 50000.00, 1);  
19  
20 INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)  
21 VALUES (102, 'PRATIK', 'bob@example.com', 60000.00, 2);  
22  
23 INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)  
24 VALUES (103, 'Charlie', 'charlie@example.com', 55000.00, 1);  
25  
26 INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)  
27 VALUES (104, 'David', 'david@example.com', 45000.00, 2);  
28  
29 INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)  
30 VALUES (105, 'Frank', 'frank@example.com', 40000.00, 1);  
31  
32 SELECT * FROM Employee;  
33 SELECT * FROM Department;  
34
```

The right pane shows the output of the queries:

Output:

EmpID	Name	Email	Salary	DeptID
101	sudarshan	sudarshan.d@example.com	50000.00	1
102	PRATIK	bob@example.com	60000.00	2
103	Charlie	charlie@example.com	55000.00	1
104	David	david@example.com	45000.00	2
105	Frank	frank@example.com	40000.00	1

DeptID	DeptName
1	HR
2	IT

Lab File – Practical 5

Aim: Create a table for Customer details with various integrity constraints like NOT NULL, CHECK, and DEFAULT. Insert valid and invalid data to test these constraints and ensure data integrity.

Code :

```
CREATE TABLE Department (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50) UNIQUE  
);  
  
-- Create Employee Table  
  
CREATE TABLE Employee (  
    EmpID INT PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) UNIQUE,  
    Salary DECIMAL(10,2) CHECK (Salary > 0),  
    DeptID INT,  
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
);  
  
INSERT INTO Department (DeptID, DeptName) VALUES (1, 'HR');  
INSERT INTO Department (DeptID, DeptName) VALUES (2, 'IT');  
  
INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)  
VALUES (101, 'sudarshan', 'sudarshan.d@example.com', 50000.00, 1);  
  
INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)  
VALUES (102, 'PRATIK', 'bob@example.com', 60000.00, 2);  
  
INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)  
VALUES (103, 'Charlie', 'charlie@example.com', 55000.00, 1);
```



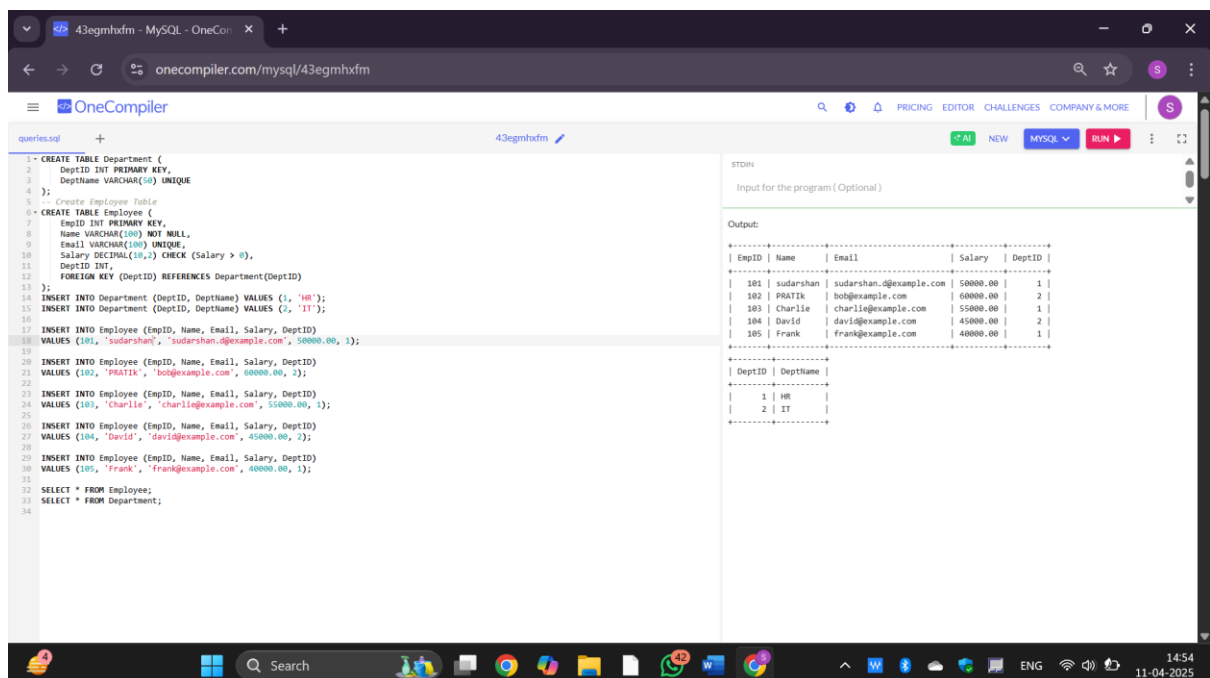
```
INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)
VALUES (104, 'David', 'david@example.com', 45000.00, 2);
```

```
INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)
VALUES (105, 'Frank', 'frank@example.com', 40000.00, 1);
```

```
SELECT * FROM Employee;
```

```
SELECT * FROM Department;
```

Output :



The screenshot shows the OneCompiler MySQL IDE interface. The left pane contains a SQL script with the following queries:

```
1- CREATE TABLE Department (
2   DeptID INT PRIMARY KEY,
3   DeptName VARCHAR(50) UNIQUE
4 );
5 -- Create Employee Table
6- CREATE TABLE Employee (
7   EmpID INT PRIMARY KEY,
8   Name VARCHAR(100) NOT NULL,
9   Email VARCHAR(100) UNIQUE,
10  Salary DECIMAL(10,2) CHECK (Salary > 0),
11  DeptID INT,
12  FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
13 );
14 INSERT INTO Department (DeptID, DeptName) VALUES (1, 'HR');
15 INSERT INTO Department (DeptID, DeptName) VALUES (2, 'IT');
16
17 INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)
18 VALUES (101, 'Sudarshan', 'sudarshan.d@example.com', 50000.00, 1);
19
20 INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)
21 VALUES (102, 'Pratik', 'bob@example.com', 60000.00, 2);
22
23 INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)
24 VALUES (103, 'Charlie', 'charlie@example.com', 55000.00, 1);
25
26 INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)
27 VALUES (104, 'David', 'david@example.com', 45000.00, 2);
28
29 INSERT INTO Employee (EmpID, Name, Email, Salary, DeptID)
30 VALUES (105, 'Frank', 'frank@example.com', 40000.00, 1);
31
32 SELECT * FROM Employee;
33 SELECT * FROM Department;
```

The right pane shows the output of the queries. The first query, `SELECT * FROM Employee;`, returns the following table:

EmpID	Name	Email	Salary	DeptID
101	Sudarshan	sudarshan.d@example.com	50000.00	1
102	Pratik	bob@example.com	60000.00	2
103	Charlie	charlie@example.com	55000.00	1
104	David	david@example.com	45000.00	2
105	Frank	frank@example.com	40000.00	1

The second query, `SELECT * FROM Department;`, returns the following table:

DeptID	DeptName
1	HR
2	IT

Lab File – Practical 6 :

Aim: Use DDL commands to create tables and DML commands to insert, update, and delete data. Write SELECT queries to retrieve and verify data changes.

Code :

-- Create Table

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    FirstName VARCHAR(50),  
    LastName VARCHAR(50),  
    Age INT,  
    Department VARCHAR(50),  
    Salary DECIMAL(10, 2)  
);
```

-- DML Commands - Insert Data

```
INSERT INTO Employees (EmployeeID, FirstName, LastName, Age, Department, Salary)  
VALUES (1, 'sudarshan', 'dharam', 28, 'HR', 50000.00);
```

```
INSERT INTO Employees (EmployeeID, FirstName, LastName, Age, Department, Salary)  
VALUES (2, 'Jane', 'Smith', 35, 'IT', 65000.00);
```

```
INSERT INTO Employees (EmployeeID, FirstName, LastName, Age, Department, Salary)  
VALUES (3, 'Michael', 'Johnson', 40, 'Finance', 75000.00);
```

-- 1. Update a single column (e.g., update salary for EmployeeID 2)

```
UPDATE Employees  
SET Salary = 70000.00  
WHERE EmployeeID = 2;
```

-- 2. Update multiple columns for a specific row (e.g., update name and salary for EmployeeID 2)

```
UPDATE Employees
```

```
SET FirstName = 'sudarshan', LastName = 'Dharam', Salary = 75000.00
```

```
WHERE EmployeeID = 2;
```

```
-- 3. Update entire tuple (all columns for EmployeeID 3)
```

```
UPDATE Employees
```

```
SET FirstName = 'Michael', LastName = 'Brown', Age = 45, Department = 'Management', Salary = 80000.00
```

```
WHERE EmployeeID = 3;
```

```
-- 4. Update with a condition (e.g., increase salary by 10% for all employees in HR)
```

```
UPDATE Employees
```

```
SET Salary = Salary * 1.10
```

```
WHERE Department = 'HR';
```

```
-- 5. Update with a subquery (increase salary for Employee with highest salary)
```

```
UPDATE Employees
```

```
SET Salary = Salary + 5000
```

```
WHERE Salary = (
```

```
    SELECT max_sal FROM (
```

```
        SELECT MAX(Salary) AS max_sal FROM Employees
```

```
    ) AS temp
```

```
);
```

```
-- 6. Update using a CASE statement (e.g., increase salary based on department)
```

```
UPDATE Employees
```

```
SET Salary = CASE
```

```
    WHEN Department = 'HR' THEN Salary * 1.05
```

```
    WHEN Department = 'IT' THEN Salary * 1.08
```

```
    WHEN Department = 'Finance' THEN Salary * 1.10
```

```
    ELSE Salary
```

```
END;
```

```
-- Delete Data from the Table
```

```
DELETE FROM Employees
```

```
WHERE EmployeeID = 1;
```

-- Select and Verify Data

-- Retrieve all data

```
SELECT * FROM Employees;
```

-- Check update for EmployeeID = 2

```
SELECT * FROM Employees
```

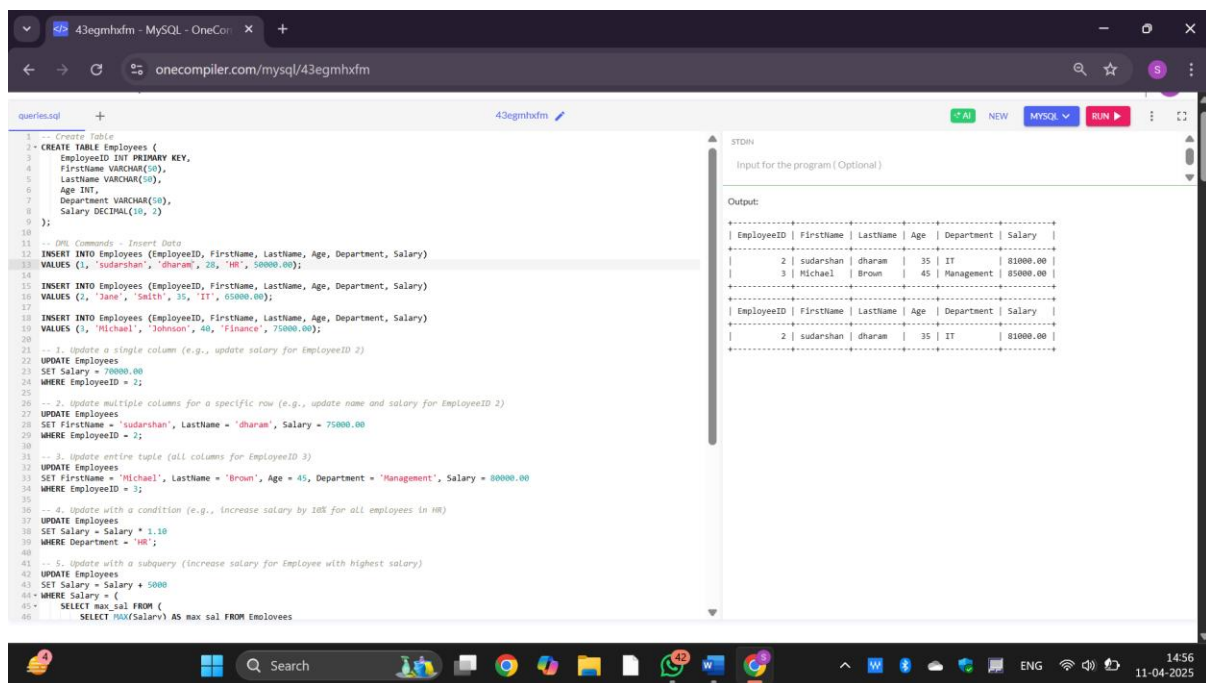
```
WHERE EmployeeID = 2;
```

-- Check if EmployeeID = 1 still exists

```
SELECT * FROM Employees
```

```
WHERE EmployeeID = 1;
```

Output :



The screenshot shows a web-based MySQL IDE interface. The left pane contains a list of SQL queries, and the right pane shows the output of the selected query. The queries include creating a table, inserting data, and various update operations. The output shows the result of the first query, which is a table with 5 columns: EmployeeID, FirstName, LastName, Age, and Salary.

```
1 -- Create Table
2 CREATE TABLE Employees (
3   EmployeeID INT PRIMARY KEY,
4   FirstName VARCHAR(10),
5   LastName VARCHAR(10),
6   Age INT,
7   Department VARCHAR(10),
8   Salary DECIMAL(10, 2)
9 );
10
11 -- DML Commands - Insert Data
12 INSERT INTO Employees (EmployeeID, FirstName, LastName, Age, Department, Salary)
13 VALUES (1, 'Sudashan', 'Dharam', 28, 'HR', 50000.00);
14
15 INSERT INTO Employees (EmployeeID, FirstName, LastName, Age, Department, Salary)
16 VALUES (2, 'Jane', 'Smith', 35, 'IT', 65000.00);
17
18 INSERT INTO Employees (EmployeeID, FirstName, LastName, Age, Department, Salary)
19 VALUES (3, 'Michael', 'Johnson', 40, 'Finance', 75000.00);
20
21 -- 1. Update a single column (e.g., update salary for EmployeeID 2)
22 UPDATE Employees
23 SET Salary = 70000.00
24 WHERE EmployeeID = 2;
25
26 -- 2. Update multiple columns for a specific row (e.g., update name and salary for EmployeeID 2)
27 UPDATE Employees
28 SET FirstName = 'Sudashan', LastName = 'Dharam', Salary = 75000.00
29 WHERE EmployeeID = 2;
30
31 -- 3. Update entire tuple (all columns for EmployeeID 3)
32 UPDATE Employees
33 SET FirstName = 'Michael', LastName = 'Brown', Age = 45, Department = 'Management', Salary = 80000.00
34 WHERE EmployeeID = 3;
35
36 -- 4. Update with a condition (e.g., increase salary by 10% for all employees in HR)
37 UPDATE Employees
38 SET Salary = Salary * 1.10
39 WHERE Department = 'HR';
40
41 -- 5. Update with a subquery (Increase salary for Employee with highest salary)
42 UPDATE Employees
43 SET Salary = Salary + 5000
44 WHERE Salary = (
45   SELECT max_sal FROM (
46     SELECT MAX(Salary) AS max_sal FROM Employees
```

Output:

EmployeeID	FirstName	LastName	Age	Department	Salary
1	Sudashan	Dharam	28	HR	50000.00
2	Sudashan	Dharam	35	IT	81000.00
3	Michael	Brown	45	Management	85000.00

Lab File – Practical 7

Aim : Create a Sales table and use aggregate functions like COUNT, SUM, AVG, MIN, and MAX to summarize sales data and calculate statistics.

Code :

-- Create the Sales table

```
CREATE TABLE Sales (  
    SaleID INT PRIMARY KEY AUTO_INCREMENT,  
    Product VARCHAR(50),  
    Quantity INT,  
    Price DECIMAL(10,2),  
    SaleDate DATE  
);
```

-- Insert valid sales data

```
INSERT INTO Sales (Product, Quantity, Price, SaleDate) VALUES  
( 'Laptop', 2, 75000.00, '2025-02-01'),  
( 'Mobile', 5, 20000.00, '2025-02-02'),  
( 'Tablet', 3, 30000.00, '2025-02-03'),  
( 'Laptop', 1, 78000.00, '2025-02-04'),  
( 'Mobile', 4, 22000.00, '2025-02-05'),  
( 'Tablet', 2, 32000.00, '2025-02-06');
```

-- COUNT queries

```
SELECT COUNT(*) AS Total_Sales FROM Sales;  
  
SELECT COUNT(DISTINCT Product) AS Unique_Products FROM Sales;  
  
SELECT Product, COUNT(*) AS Sales_Count FROM Sales GROUP BY Product;  
  
SELECT SaleDate, COUNT(*) AS Sales_Per_Day FROM Sales GROUP BY SaleDate;  
  
SELECT COUNT(*) AS High_Quantity_Sales FROM Sales WHERE Quantity > 2;  
  
SELECT COUNT(*) AS Sales_This_Month  
FROM Sales  
WHERE MONTH(SaleDate) = MONTH(CURRENT_DATE)  
AND YEAR(SaleDate) = YEAR(CURRENT_DATE);  
  
SELECT COUNT(*) AS High_Value_Sales FROM Sales WHERE (Quantity * Price) > 50000;  
  
SELECT Product, COUNT(*) AS High_Value_Transactions  
FROM Sales  
WHERE (Quantity * Price) > 40000
```

```

GROUP BY Product;

SELECT COUNT(*) AS Sales_After_Date FROM Sales WHERE SaleDate > '2025-02-03';


-- SUM queries

SELECT SUM(Quantity * Price) AS Total_Revenue FROM Sales;

SELECT SUM(Quantity) AS Total_Quantity_Sold FROM Sales;

SELECT Product, SUM(Quantity * Price) AS Revenue_Per_Product FROM Sales GROUP BY Product;

SELECT SaleDate, SUM(Quantity * Price) AS Revenue_Per_Day FROM Sales GROUP BY SaleDate;

SELECT SUM(Quantity * Price) AS Revenue_This_Month

FROM Sales

WHERE MONTH(SaleDate) = MONTH(CURRENT_DATE)

    AND YEAR(SaleDate) = YEAR(CURRENT_DATE);

SELECT SUM(Quantity * Price) AS High_Quantity_Revenue FROM Sales WHERE Quantity > 2;

SELECT SUM(Quantity * Price) AS Revenue_After_Date FROM Sales WHERE SaleDate > '2025-02-03';

SELECT Product, SUM(Quantity * Price) AS High_Value_Revenue

FROM Sales

WHERE (Quantity * Price) > 40000

GROUP BY Product;


-- AVG queries

SELECT AVG(Price) AS Average_Price FROM Sales;

SELECT AVG(Quantity) AS Average_Quantity_Sold FROM Sales;

SELECT AVG(Quantity * Price) AS Average_Revenue_Per_Transaction FROM Sales;

SELECT Product, AVG(Price) AS Average_Price_Per_Product FROM Sales GROUP BY Product;

SELECT Product, AVG(Quantity * Price) AS Average_Revenue_Per_Product FROM Sales GROUP BY Product;

SELECT Product, AVG(Quantity) AS Average_Quantity_Per_Product FROM Sales GROUP BY Product;

SELECT SaleDate, AVG(Quantity * Price) AS Average_Revenue_Per_Day FROM Sales GROUP BY SaleDate;

SELECT AVG(Quantity * Price) AS Average_Revenue_This_Month

FROM Sales

WHERE MONTH(SaleDate) = MONTH(CURRENT_DATE)

    AND YEAR(SaleDate) = YEAR(CURRENT_DATE);

SELECT AVG(Price) AS Avg_Price_High_Quantity_Sales FROM Sales WHERE Quantity > 2;

SELECT AVG(Quantity * Price) AS Average_Revenue_After_Date FROM Sales WHERE SaleDate > '2025-02-03';

```

-- MIN and MAX queries

```
SELECT MIN(Price) AS Min_Price, MAX(Price) AS Max_Price FROM Sales;
```

```
SELECT MIN(Quantity) AS Min_Quantity_Sold, MAX(Quantity) AS Max_Quantity_Sold FROM Sales;
```

```
SELECT MIN(Quantity * Price) AS Min_Revenue, MAX(Quantity * Price) AS Max_Revenue FROM Sales;
```

```
SELECT Product, MIN(Price) AS Min_Price_Per_Product, MAX(Price) AS Max_Price_Per_Product FROM Sales  
GROUP BY Product;
```

```
SELECT Product, MIN(Quantity * Price) AS Min_Revenue_Per_Product, MAX(Quantity * Price) AS  
Max_Revenue_Per_Product FROM Sales GROUP BY Product;
```

```
SELECT Product, MIN(Quantity) AS Min_Quantity_Per_Product, MAX(Quantity) AS Max_Quantity_Per_Product  
FROM Sales GROUP BY Product;
```

```
SELECT SaleDate, MIN(Quantity * Price) AS Min_Revenue_Per_Day, MAX(Quantity * Price) AS  
Max_Revenue_Per_Day FROM Sales GROUP BY SaleDate;
```

```
SELECT MIN(Quantity * Price) AS Min_Revenue_This_Month, MAX(Quantity * Price) AS  
Max_Revenue_This_Month
```

```
FROM Sales
```

```
WHERE MONTH(SaleDate) = MONTH(CURRENT_DATE)
```

```
AND YEAR(SaleDate) = YEAR(CURRENT_DATE);
```

```
SELECT MIN(Price) AS Min_Price_High_Quantity_Sales, MAX(Price) AS Max_Price_High_Quantity_Sales FROM  
Sales WHERE Quantity > 2;
```

```
SELECT MIN(Quantity * Price) AS Min_Revenue_After_Date, MAX(Quantity * Price) AS  
Max_Revenue_After_Date FROM Sales WHERE SaleDate > '2025-02-03';
```

Output :

The screenshot displays the OneCompiler MySQL IDE interface. The left pane contains a series of SQL queries for creating a table, inserting data, and performing various COUNT and SUM operations. The right pane shows the output of these queries, including total sales, unique products, sales by product, sales by date, and high-value transactions.

```
1 -- Create the Sales table
2 CREATE TABLE Sales (
3   SaleID INT PRIMARY KEY AUTO_INCREMENT,
4   Product VARCHAR(50),
5   Quantity INT,
6   Price DECIMAL(10,2),
7   SaleDate DATE
8 );
9
10 -- Insert valid sales data
11 INSERT INTO Sales (Product, Quantity, Price, SaleDate) VALUES
12 ('Laptop', 2, 75000.00, '2025-02-01'),
13 ('Mobile', 5, 20000.00, '2025-02-02'),
14 ('Tablet', 3, 30000.00, '2025-02-03'),
15 ('Laptop', 1, 78000.00, '2025-02-04'),
16 ('Mobile', 4, 22000.00, '2025-02-05'),
17 ('Tablet', 2, 32000.00, '2025-02-06');
18
19 -- COUNT queries
20 SELECT COUNT(*) AS Total_Sales FROM Sales;
21 SELECT COUNT(DISTINCT Product) AS Unique_Products FROM Sales;
22 SELECT Product, COUNT(*) AS Sales_Count FROM Sales GROUP BY Product;
23 SELECT SaleDate, COUNT(*) AS Sales_Per_Day FROM Sales GROUP BY SaleDate;
24 SELECT COUNT(*) AS High_Quantity_Sales FROM Sales WHERE Quantity > 2;
25 SELECT COUNT(*) AS Sales_This_Month
26 FROM Sales
27 WHERE MONTH(SaleDate) = MONTH(CURRENT_DATE);
28 AND YEAR(SaleDate) = YEAR(CURRENT_DATE);
29 SELECT COUNT(*) AS High_Value_Sales FROM Sales WHERE (Quantity * Price) >
30 SELECT Product, COUNT(*) AS High_Value_Transactions
31 FROM Sales
32 WHERE (Quantity * Price) > 40000
33 GROUP BY Product;
34 SELECT COUNT(*) AS Sales_After_Date FROM Sales WHERE SaleDate > '2025-02-0
35
36 -- SUM queries
37 SELECT SUM(Quantity * Price) AS Total_Revenue FROM Sales;
38 SELECT SUM(Quantity) AS Total_Quantity_Sold FROM Sales;
39 SELECT Product, SUM(Quantity * Price) AS Revenue_Per_Product FROM Sales GR
40 SELECT SaleDate, SUM(Quantity * Price) AS Revenue_Per_Day FROM Sales GROUP
41 FROM Sales
42 SELECT SUM(Quantity * Price) AS Revenue_This_Month
43 FROM Sales
44 WHERE MONTH(SaleDate) = MONTH(CURRENT_DATE);
45 AND YEAR(SaleDate) = YEAR(CURRENT_DATE);
```

Output:

Total_Sales	
	6

Unique_Products	
	3

Product	Sales_Count
Laptop	2
Mobile	2
Tablet	2

SaleDate	Sales_Per_Day
2025-02-01	1
2025-02-02	1
2025-02-03	1
2025-02-04	1
2025-02-05	1
2025-02-06	1

High_Quantity_Sales	
---------------------	--

https://onecompiler.com/mysql/43dpxpky

OneCompiler

queries.sql +

```
1 -- Create the Sales table
2 CREATE TABLE Sales (
3     SaleId INT PRIMARY KEY AUTO_INCREMENT,
4     Product VARCHAR(50),
5     Quantity INT,
6     Price DECIMAL(10,2),
7     SaleDate DATE
8 );
9
10 -- Insert valid sales data
11 INSERT INTO Sales (Product, Quantity, Price, SaleDate) VALUES
12 ('Laptop', 2, 75000.00, '2025-02-01'),
13 ('Mobile', 5, 20000.00, '2025-02-02'),
14 ('Tablet', 3, 30000.00, '2025-02-03'),
15 ('Laptop', 1, 78000.00, '2025-02-04'),
16 ('Mobile', 4, 22000.00, '2025-02-05'),
17 ('Tablet', 2, 32000.00, '2025-02-06');
18
19 -- COUNT queries
20 SELECT COUNT(*) AS Total_Sales FROM Sales;
21 SELECT COUNT(DISTINCT Product) AS Unique_Products FROM Sales;
22 SELECT Product, COUNT(*) AS Sales_Count FROM Sales GROUP BY Product;
23 SELECT SaleDate, COUNT(*) AS Sales_Per_Day FROM Sales GROUP BY SaleDate;
24 SELECT COUNT(*) AS High_Quantity_Sales FROM Sales WHERE Quantity > 2;
25 SELECT COUNT(*) AS Sales_This_Month
26 FROM Sales
27 WHERE MONTH(SaleDate) = MONTH(CURRENT_DATE)
28 AND YEAR(SaleDate) = YEAR(CURRENT_DATE);
29 SELECT COUNT(*) AS High_Value_Sales FROM Sales WHERE (Quantity * Price) > 50000;
30 SELECT Product, COUNT(*) AS High_Value_Transactions
31 FROM Sales
32 WHERE (Quantity * Price) > 40000
33 GROUP BY Product;
34 SELECT COUNT(*) AS Sales_After_Date FROM Sales WHERE SaleDate > '2025-02-03';
35
36 -- SUM queries
37 SELECT SUM(Quantity * Price) AS Total_Revenue FROM Sales;
38 SELECT SUM(Quantity) AS Total_Quantity_Sold FROM Sales;
39 SELECT Product, SUM(Quantity * Price) AS Revenue_Per_Product FROM Sales GROUP BY Product;
40 SELECT SaleDate, SUM(Quantity * Price) AS Revenue_Per_Day FROM Sales GROUP BY SaleDate;
41 SELECT SUM(Quantity * Price) AS Revenue_This_Month
42 FROM Sales
43 WHERE MONTH(SaleDate) = MONTH(CURRENT_DATE)
44 AND YEAR(SaleDate) = YEAR(CURRENT_DATE);
45 SELECT SUM(Quantity * Price) AS High_Quantity_Revenue FROM Sales WHERE Quantity > 2;
46 SELECT SUM(Quantity * Price) AS Revenue_After_Date FROM Sales WHERE SaleDate > '2025-02-03';
47 SELECT Product, SUM(Quantity * Price) AS High_Value_Revenue
48 FROM Sales
49 WHERE (Quantity * Price) > 40000
50 GROUP BY Product;
51
52 -- AVG queries
53 SELECT AVG(Price) AS Average_Price FROM Sales;
54 SELECT AVG(Quantity) AS Average_Quantity_Sold FROM Sales;
55 SELECT AVG(Quantity * Price) AS Average_Revenue_Per_Transaction FROM Sales;
56 SELECT Product, AVG(Price) AS Average_Price_Per_Product FROM Sales GROUP BY Product;
57 SELECT Product, AVG(Quantity * Price) AS Average_Revenue_Per_Product FROM Sales GROUP BY Product;
58 SELECT Product, AVG(Quantity) AS Average_Quantity_Per_Product FROM Sales GROUP BY Product;
```

43dpxpky

STON

Input for the program (Optional)

Total_Revenue	159000.00
Total_Quantity_Sold	17
Revenue_Per_Product	
Laptop	228000.00
Mobile	108000.00
Tablet	154000.00
Revenue_Per_Day	
2025-02-01	158000.00
2025-02-02	109000.00
2025-02-03	90000.00
2025-02-04	78000.00
2025-02-05	88000.00
2025-02-06	64000.00
Revenue_This_Month	
	NULL
High_Quantity_Revenue	
	278000.00
Average_Price	
	23000.00
Average_Quantity_Sold	
	2.8333
Average_Revenue_Per_Transaction	
	95000.000000
Average_Price_Per_Product	
Laptop	75000.000000
Mobile	21800.000000
Tablet	31000.000000
Average_Revenue_Per_Product	
Laptop	114000.000000
Mobile	94000.000000
Tablet	77000.000000
Average_Quantity_Per_Product	
Laptop	1.5000
Mobile	4.5000
Tablet	2.5000
Average_Revenue_Per_Day	
2025-02-01	158000.000000

OneCompiler

queries.sql

```
1 -- Create the Sales table
2 * CREATE TABLE Sales (
3   SaleID INT PRIMARY KEY AUTO_INCREMENT,
4   Product VARCHAR(50),
5   Quantity INT,
6   Price DECIMAL(10,2),
7   SaleDate DATE
8 );
9
10 -- Insert valid sales data
11 INSERT INTO Sales (Product, Quantity, Price, SaleDate) VALUES
12 ('Laptop', 2, 75000.00, '2025-02-01'),
13 ('Mobile', 5, 20000.00, '2025-02-02'),
14 ('Tablet', 3, 30000.00, '2025-02-03'),
15 ('Laptop', 1, 78000.00, '2025-02-04'),
16 ('Mobile', 4, 22000.00, '2025-02-05'),
17 ('Tablet', 2, 32000.00, '2025-02-06');
18
19 -- COUNT queries
20 SELECT COUNT(*) AS Total_Sales FROM Sales;
21 SELECT COUNT(DISTINCT Product) AS Unique_Products FROM Sales;
22 SELECT Product, COUNT(*) AS Sales_Count FROM Sales GROUP BY Product;
23 SELECT SaleDate, COUNT(*) AS Sales_Per_Day FROM Sales GROUP BY SaleDate;
24 SELECT COUNT(*) AS High_Quantity_Sales FROM Sales WHERE Quantity > 2;
25 SELECT COUNT(*) AS Sales_This_Month
26 FROM Sales
27 WHERE MONTH(SaleDate) = MONTH(CURRENT_DATE)
28 AND YEAR(SaleDate) = YEAR(CURRENT_DATE);
29 SELECT COUNT(*) AS High_Value_Sales FROM Sales WHERE (Quantity * Price) > 50000;
30 SELECT Product, COUNT(*) AS High_Value_Transactions
31 FROM Sales
32 WHERE (Quantity * Price) > 40000
33 GROUP BY Product;
34 SELECT COUNT(*) AS Sales_After_Date FROM Sales WHERE SaleDate > '2025-02-03';
35
36 -- SUM queries
37 SELECT SUM(Quantity * Price) AS Total_Revenue FROM Sales;
38 SELECT SUM(Quantity) AS Total_Quantity_Sold FROM Sales;
39 SELECT Product, SUM(Quantity * Price) AS Revenue_Per_Product FROM Sales GROUP BY Product;
40 SELECT SaleDate, SUM(Quantity * Price) AS Revenue_Per_Day FROM Sales GROUP BY SaleDate;
41 SELECT SUM(Quantity * Price) AS Revenue_This_Month
42 FROM Sales
43 WHERE MONTH(SaleDate) = MONTH(CURRENT_DATE)
44 AND YEAR(SaleDate) = YEAR(CURRENT_DATE);
45 SELECT SUM(Quantity * Price) AS High_Quantity_Revenue FROM Sales WHERE Quantity > 2;
46 SELECT SUM(Quantity * Price) AS Revenue_After_Date FROM Sales WHERE SaleDate > '2025-02-03';
47 SELECT Product, SUM(Quantity * Price) AS High_Value_Revenue
48 FROM Sales
49 WHERE (Quantity * Price) > 40000
50 GROUP BY Product;
51
52 -- AVG queries
53 SELECT AVG(Price) AS Average_Price FROM Sales;
54 SELECT AVG(Quantity) AS Average_Quantity_Sold FROM Sales;
55 SELECT AVG(Quantity * Price) AS Average_Revenue_Per_Transaction FROM Sales;
56 SELECT Product, AVG(Price) AS Average_Price_Per_Product FROM Sales GROUP BY Product;
57 SELECT Product, AVG(Quantity * Price) AS Average_Revenue_Per_Product FROM Sales GROUP BY Product;
58 SELECT Product, AVG(Quantity) AS Average_Quantity_Per_Product FROM Sales GROUP BY Product;
```

STON

Input for the program (Optional)

SaleDate	Average_Revenue_Per_Day
2025-02-01	150000.000000
2025-02-02	100000.000000
2025-02-03	90000.000000
2025-02-04	78000.000000
2025-02-05	88000.000000
2025-02-06	64000.000000

Average_Revenue_This_Month
76666.666667

Avg_Price_High_Quantity_Sales
20000.00
78000.00

Min_Price	Max_Price
20000.00	78000.00

Min_Quantity_Sold	Max_Quantity_Sold
1	5

Min_Revenue	Max_Revenue
64000.00	150000.00

Product	Min_Price_Per_Product	Max_Price_Per_Product
Laptop	75000.00	78000.00
Mobile	20000.00	22000.00
Tablet	30000.00	32000.00

Product	Min_Revenue_Per_Product	Max_Revenue_Per_Product
Laptop	78000.00	150000.00
Mobile	88000.00	100000.00
Tablet	64000.00	90000.00

Product	Min_Quantity_Per_Product	Max_Quantity_Per_Product
Laptop	1	2
Mobile	4	5
Tablet	2	3

SaleDate	Min_Revenue_Per_Day	Max_Revenue_Per_Day
2025-02-01	150000.00	150000.00
2025-02-02	100000.00	100000.00
2025-02-03	90000.00	90000.00
2025-02-04	78000.00	78000.00
2025-02-05	88000.00	88000.00
2025-02-06	64000.00	64000.00

Min_Revenue_This_Month	Max_Revenue_This_Month
NULL	NULL

Min_Price_High_Quantity_Sales	Max_Price_High_Quantity_Sales
20000.00	30000.00

Min_Revenue_After_Date	Max_Revenue_After_Date
64000.00	88000.00

Lab File – Practical 8 :

Aim : Given Customers and Orders tables, write SQL queries to perform INNER JOIN, LEFT JOIN, and RIGHT JOIN to retrieve combined data for customer orders.

Code :

-- Create Customers table

CREATE TABLE Customers (

customer id INT PRIMARY KEY,

customer name VARCHAR(100) NOT NULL

);

-- Create Orders table with foreign key referencing Customers

CREATE TABLE Orders (

order id INT PRIMARY KEY,

order date DATE NOT NULL,

customer id INT,

FOREIGN KEY (customer id) REFERENCES

Customers(customer id)

);

-- Insert into Customers

INSERT INTO Customers (customer id, customer name) VALUES

(1, 'sudarshan'),

(2, 'Bob'),

(3, 'Charlie'),

(4, 'David');

-- Insert into Orders

INSERT INTO Orders (order_id, order_date, customer_id) VALUES

(101, '2024-01-01', 1),

(102, '2024-01-02', 2),

(103, '2024-01-03', 4);

-- Show Customers table

SELECT * FROM Customers;

-- Show Orders table

SELECT * FROM Orders;

-- INNER JOIN - Only matching customer & order records

SELECT

c.customer_id,

c.customer_name,

o.order_id,

o.order_date

FROM

Customers c

INNER JOIN

Orders o

ON

c.customer_id = o.customer_id;

-- LEFT JOIN - All Customers with their Orders (if any)

SELECT

c.customer_id,

c.customer_name,

o.order_id,

o.order_date

FROM

Customers c

LEFT JOIN

Orders o

ON

c.customer_id = o.customer_id;

-- RIGHT JOIN - All Orders with Customer details (if any)

SELECT

c.customer_id,

c.customer_name,

o.order_id,

o.order_date

FROM

Customers c

RIGHT JOIN

Orders o

ON

c.customer_id = o.customer_id;

output :

The screenshot shows the OneCompiler MySQL interface. The SQL code in the editor is as follows:

```
1 -- Create Customers table
2 CREATE TABLE Customers (
3   customer_id INT PRIMARY KEY,
4   customer_name VARCHAR(100) NOT NULL
5 );
6
7 -- Create Orders table with foreign key referencing Customers
8 CREATE TABLE Orders (
9   order_id INT PRIMARY KEY,
10  order_date DATE NOT NULL,
11  customer_id INT,
12  FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
13 );
14
15 -- Insert into Customers
16 INSERT INTO Customers (customer_id, customer_name) VALUES
17 (1, 'sudarshan'),
18 (2, 'Bob'),
19 (3, 'Charlie'),
20 (4, 'David');
21
22 -- Insert into Orders
23 INSERT INTO Orders (order_id, order_date, customer_id) VALUES
24 (101, '2024-01-01', 1),
25 (102, '2024-01-02', 2),
26 (103, '2024-01-03', 4);
27
28 -- Show Customers table
29 SELECT * FROM Customers;
30
31 -- Show Orders table
32 SELECT * FROM Orders;
33
34 -- INNER JOIN - Only matching customer & order records
35 SELECT
36   c.customer_id,
37   c.customer_name,
38   o.order_id,
39   o.order_date
40 FROM
41   Customers c
42 INNER JOIN
43   Orders o
44 ON
45   c.customer_id = o.customer_id;
```

The output of the queries is displayed on the right side of the interface:

Input for the program (Optional)

Output:

customer_id	customer_name
1	sudarshan
2	Bob
3	Charlie
4	David

order_id	order_date	customer_id
101	2024-01-01	1
102	2024-01-02	2
103	2024-01-03	4

customer_id	customer_name	order_id	order_date
1	sudarshan	101	2024-01-01
2	Bob	102	2024-01-02
4	David	103	2024-01-03

customer_id	customer_name	order_id	order_date
1	sudarshan	101	2024-01-01
2	Bob	102	2024-01-02
3	Charlie	NULL	NULL
4	David	103	2024-01-03

43egmhxfm - MySQL - OneCompiler

onecompiler.com/mysql/43egmhxfm

queries.sql

```

1  -- Create Customers table
2  CREATE TABLE Customers (
3      customer_id INT PRIMARY KEY,
4      customer_name VARCHAR(100) NOT NULL
5  );
6
7  -- Create Orders table with foreign key referencing Customers
8  CREATE TABLE Orders (
9      order_id INT PRIMARY KEY,
10     order_date DATE NOT NULL,
11     customer_id INT,
12     FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
13 );
14
15 -- Insert into Customers
16 INSERT INTO Customers (customer_id, customer_name) VALUES
17 (1, 'Sudarshan'),
18 (2, 'Bob'),
19 (3, 'Charlie'),
20 (4, 'David');
21
22 -- Insert into Orders
23 INSERT INTO Orders (order_id, order_date, customer_id) VALUES
24 (101, '2024-01-01', 1),
25 (102, '2024-01-02', 2),
26 (103, '2024-01-03', 4);
27
28 -- Show Customers table
29 SELECT * FROM Customers;
30
31 -- Show Orders table
32 SELECT * FROM Orders;
33
34 -- INNER JOIN - Only matching customer & order records
35 SELECT
36     c.customer_id,
37     c.customer_name,
38     o.order_id,
39     o.order_date
40 FROM
41     Customers c
42 INNER JOIN
43     Orders o
44 ON
45     c.customer_id = o.customer_id;

```

STDIN

Input for the program (Optional)

order_id	order_date	customer_id
1	2024-01-01	1
2	2024-01-02	2
3	2024-01-03	4

customer_id	customer_name	order_id	order_date
1	Sudarshan	101	2024-01-01
2	Bob	102	2024-01-02
4	David	103	2024-01-03

15:04 11-04-2025